

IFJ25

Generováno programem Doxygen 1.9.1

1 README	1
2 Rejstřík datových struktur	3
2.1 Datové struktury	3
3 Rejstřík souborů	5
3.1 Seznam souborů	5
4 Dokumentace datových struktur	7
4.1 Dokumentace struktury AstNode	7
4.1.1 Detailní popis	7
4.2 Dokumentace struktury AstNodeAssignStmt	7
4.3 Dokumentace struktury AstNodeBinaryExpr	8
4.4 Dokumentace struktury AstNodeBlock	8
4.5 Dokumentace struktury AstNodeFuncCall	8
4.6 Dokumentace struktury AstNodeFuncDef	9
4.7 Dokumentace struktury AstNodeIfStmt	9
4.8 Dokumentace struktury AstNodeList	9
4.8.1 Detailní popis	10
4.9 Dokumentace struktury AstNodeLiteral	10
4.10 Dokumentace struktury AstNodeProgram	10
4.11 Dokumentace struktury AstNodeReturnStmt	10
4.12 Dokumentace struktury AstNodeVarDecl	11
4.13 Dokumentace struktury AstNodeVariable	11
4.14 Dokumentace struktury AstNodeWhileStmt	11
4.15 Dokumentace struktury LexemeBuffer	12
4.15.1 Detailní popis	12
4.16 Dokumentace struktury stack_node	12
4.16.1 Detailní popis	13
4.17 Dokumentace struktury symbol_data	13
4.17.1 Detailní popis	13
4.18 Dokumentace struktury symstack	13
4.18.1 Detailní popis	14
4.19 Dokumentace struktury Token	14
4.19.1 Detailní popis	14
4.20 Dokumentace struktury tree_node	14
4.20.1 Detailní popis	15
4.21 Dokumentace unie Token::value	15
5 Dokumentace souborů	17
5.1 Dokumentace souboru ast.c	17
5.2 Dokumentace souboru ast.h	17
5.2.1 Detailní popis	19
5.2.2 Dokumentace výčtových typů	19

5.2.2.1 AstNodeType	19
5.2.3 Dokumentace funkcí	19
5.2.3.1 add_func_to_program()	19
5.2.3.2 add_stmt_to_block()	20
5.2.3.3 add_to_list()	20
5.2.3.4 create_assign_stmt()	20
5.2.3.5 create_binary_expr()	21
5.2.3.6 create_block()	21
5.2.3.7 create_func_call()	22
5.2.3.8 create_func_def()	22
5.2.3.9 create_if_stmt()	23
5.2.3.10 create_list()	23
5.2.3.11 create_literal()	23
5.2.3.12 create_program()	24
5.2.3.13 create_return_stmt()	24
5.2.3.14 create_vardecl()	24
5.2.3.15 create_variable()	25
5.2.3.16 create_while_stmt()	25
5.2.3.17 free_ast_node()	26
5.2.3.18 free_token_lexeme()	26
5.3 Dokumentace souboru codegen.c	26
5.3.1 Detailní popis	27
5.3.2 Dokumentace funkcí	27
5.3.2.1 generate_code()	27
5.4 Dokumentace souboru codegen.h	27
5.4.1 Detailní popis	28
5.4.2 Dokumentace funkcí	28
5.4.2.1 generate_code()	28
5.5 Dokumentace souboru err.c	28
5.5.1 Detailní popis	29
5.5.2 Dokumentace funkcí	29
5.5.2.1 ifjexit()	29
5.6 Dokumentace souboru err.h	29
5.6.1 Detailní popis	30
5.6.2 Dokumentace funkcí	30
5.6.2.1 ifjexit()	30
5.7 Dokumentace souboru lex_scanner.c	30
5.7.1 Detailní popis	31
5.7.2 Dokumentace funkcí	31
5.7.2.1 builtInFunctions()	32
5.7.2.2 createToken()	32
5.7.2.3 escapeChar()	32

5.7.2.4 ifj_get_line()	33
5.7.2.5 ifj_get_token()	33
5.7.2.6 keywords()	33
5.7.2.7 transition()	33
5.8 Dokumentace souboru lex_scanner.h	34
5.8.1 Detailní popis	35
5.8.2 Dokumentace definic typů	35
5.8.2.1 Token	35
5.8.3 Dokumentace funkcí	35
5.8.3.1 ifj_get_line()	35
5.8.3.2 ifj_get_token()	36
5.9 Dokumentace souboru main.c	36
5.9.1 Detailní popis	36
5.10 Dokumentace souboru parser.c	36
5.10.1 Detailní popis	37
5.10.2 Dokumentace k definicím maker	37
5.10.2.1 TRACEF	37
5.10.3 Dokumentace funkcí	37
5.10.3.1 get_type()	37
5.10.3.2 ifj_get_token()	38
5.10.3.3 ifj_parse_program()	38
5.10.3.4 search_scopes()	38
5.11 Dokumentace souboru parser.h	39
5.11.1 Detailní popis	39
5.11.2 Dokumentace funkcí	39
5.11.2.1 ifj_parse_program()	40
5.12 Dokumentace souboru symstack.c	40
5.12.1 Detailní popis	41
5.12.2 Dokumentace funkcí	41
5.12.2.1 stack_dispose()	41
5.12.2.2 stack_init()	41
5.12.2.3 stack_pop()	42
5.12.2.4 stack_push()	42
5.12.2.5 stack_top()	42
5.13 Dokumentace souboru symstack.h	43
5.13.1 Detailní popis	43
5.13.2 Dokumentace funkcí	43
5.13.2.1 stack_dispose()	43
5.13.2.2 stack_init()	44
5.13.2.3 stack_pop()	44
5.13.2.4 stack_push()	44
5.13.2.5 stack_top()	45

5.14 Dokumentace souboru symtable.c	45
5.14.1 Detailní popis	46
5.14.2 Dokumentace funkcí	46
5.14.2.1 balance_tree()	46
5.14.2.2 delete_symbol()	47
5.14.2.3 get_balance()	47
5.14.2.4 get_height()	47
5.14.2.5 insert_symbol()	48
5.14.2.6 put_height()	48
5.14.2.7 replace_by_rightmost()	48
5.14.2.8 rotate_left()	49
5.14.2.9 rotate_right()	49
5.14.2.10 search_symbol()	50
5.14.2.11 tree_dispose()	50
5.14.2.12 tree_init()	50
5.15 Dokumentace souboru symtable.h	51
5.15.1 Detailní popis	51
5.15.2 Dokumentace funkcí	52
5.15.2.1 delete_symbol()	52
5.15.2.2 insert_symbol()	52
5.15.2.3 search_symbol()	52
5.15.2.4 tree_dispose()	53
5.15.2.5 tree_init()	53
Rejstřík	55

Kapitola 1

README

vtipné jméno 2 2 – IFJ PROJEKT 2025

Kapitola 2

Rejstřík datových struktur

2.1 Datové struktury

Následující seznam obsahuje identifikace datových struktur a jejich stručné popisy:

AstNode	
Základní struktura každého uzlu AST	7
AstNodeAssignStmt	7
AstNodeBinaryExpr	8
AstNodeBlock	8
AstNodeFuncCall	8
AstNodeFuncDef	9
AstNodeIfStmt	9
AstNodeList	
Obecný seznam AST uzlů	9
AstNodeLiteral	10
AstNodeProgram	10
AstNodeReturnStmt	10
AstNodeVarDecl	11
AstNodeVariable	11
AstNodeWhileStmt	11
LexemeBuffer	
Struktura pro paměť lexému	12
stack_node	
Uzel v zásobníku	12
symbol_data	
Data uloženého symbolu	13
symstack	
Zásobník, ukazatel na vrchol zásobníku	13
Token	
Struktura tokenu s načteným řetězcem a jeho hodnotou	14
tree_node	
Uzel stromu	14
Token::value	15

Kapitola 3

Rejstřík souborů

3.1 Seznam souborů

Zde naleznete seznam všech dokumentovaných souborů se stručnými popisy:

ast.c	Implementace funkcí pro tvorbu a uvolnění uzlů AST	17
ast.h	Definice struktur pro abstraktní syntaktický strom (AST) jazyka IFJ25	17
codegen.c	Generování cílového kódu IFJcode25	26
codegen.h	Generování cílového kódu IFJcode25	27
err.c	Soubor obsahující jednotnou funkci pro správné vypnutí programu se správnou návratovou hodnotou	28
err.h	Hlavičkový soubor deklarující všechny návratové hodnoty	29
lex_scanner.c	Lexikální analýza	30
lex_scanner.h	Lexikální analýza	34
main.c	Hlavičkový soubor pro tabulku symbolů	36
parser.c	Syntaktická analýza (rekurzivní sestup + precedenční pro výrazy) a generování AST pro jazyk IFJ25	36
parser.h	Hlavičkový soubor pro syntaktický analyzátor (parser)	39
symstack.c	Zásobník tabulek určený k vytvoření nové tabulky symbolů pro vnořené bloky	40
symstack.h	Hlavičkový soubor pro zásobník tabulek	43
symtable.c	Tabulka symbolů implementována pomocí výškově vyváženého bin. stromu	45
symtable.h	Hlavičkový soubor pro tabulku symbolů	51

Kapitola 4

Dokumentace datových struktur

4.1 Dokumentace struktury AstNode

Základní struktura každého uzlu AST.

```
#include <ast.h>
```

Datové položky

- [AstNodeType](#) type
Typ uzlu.
- int [line_number](#)
Číslo řádku ve zdrojovém souboru.

4.1.1 Detailní popis

Základní struktura každého uzlu AST.

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.2 Dokumentace struktury AstNodeAssignStmt

Diagram tříd pro AstNodeAssignStmt:

Datové položky

- [AstNode](#) base
- [AstNodeVariable](#) * **lvalue**
- struct [AstNode](#) * **rvalue**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.3 Dokumentace struktury AstNodeBinaryExpr

Diagram tříd pro AstNodeBinaryExpr:

Datové položky

- [AstNode](#) **base**
- [TokenType](#) **op**
- struct [AstNode](#) * **left**
- struct [AstNode](#) * **right**
- [symbol_data_type](#) **data_type**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.4 Dokumentace struktury AstNodeBlock

Diagram tříd pro AstNodeBlock:

Datové položky

- [AstNode](#) **base**
- [AstNodeList](#) * **statements**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.5 Dokumentace struktury AstNodeFuncCall

Diagram tříd pro AstNodeFuncCall:

Datové položky

- [AstNode](#) **base**
- [Token](#) **func_id**
- [AstNodeList](#) * **args**
- [symbol_data_type](#) **data_type**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.6 Dokumentace struktury AstNodeFuncDef

Diagram tříd pro AstNodeFuncDef:

Datové položky

- [AstNode](#) **base**
- [Token](#) **func_id**
- [AstNodeList](#) * **params**
- [AstNodeBlock](#) * **body**
- [AstFuncKind](#) **kind**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.7 Dokumentace struktury AstNodeIfStmt

Diagram tříd pro AstNodeIfStmt:

Datové položky

- [AstNode](#) **base**
- struct [AstNode](#) * **condition**
- [AstNodeBlock](#) * **then_block**
- [AstNodeBlock](#) * **else_block**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.8 Dokumentace struktury AstNodeList

Obecný seznam AST uzlů.

```
#include <ast.h>
```

Diagram tříd pro AstNodeList:

Datové položky

- [AstNode](#) **base**
- struct [AstNode](#) ** **items**
Dynamické pole ukazatelů na uzly.
- int **count**
Aktuální počet položek.
- int **capacity**
Celková kapacita pole.

4.8.1 Detailní popis

Obecný seznam AST uzlů.

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.9 Dokumentace struktury AstNodeLiteral

Diagram tříd pro AstNodeLiteral:

Datové položky

- [AstNode](#) **base**
- [Token](#) **token**
- [symbol_data_type](#) **data_type**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.10 Dokumentace struktury AstNodeProgram

Diagram tříd pro AstNodeProgram:

Datové položky

- [AstNode](#) **base**
- [AstNodeList](#) * **functions**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.11 Dokumentace struktury AstNodeReturnStmt

Diagram tříd pro AstNodeReturnStmt:

Datové položky

- [AstNode](#) **base**
- struct [AstNode](#) * **expr**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.12 Dokumentace struktury AstNodeVarDecl

Diagram tříd pro AstNodeVarDecl:

Datové položky

- [AstNode](#) **base**
- [Token](#) **var_id**
- int **stack_offset**
- [symbol_data_type](#) **data_type**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.13 Dokumentace struktury AstNodeVariable

Diagram tříd pro AstNodeVariable:

Datové položky

- [AstNode](#) **base**
- [Token](#) **token**
- int **stack_offset**
- [symbol_data_type](#) **data_type**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.14 Dokumentace struktury AstNodeWhileStmt

Diagram tříd pro AstNodeWhileStmt:

Datové položky

- [AstNode](#) **base**
- struct [AstNode](#) * **condition**
- [AstNodeBlock](#) * **body_block**

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [ast.h](#)

4.15 Dokumentace struktury LexemeBuffer

Struktura pro paměť lexému.

Datové položky

- char * **buffer**
- size_t **length**
- size_t **capacity**

4.15.1 Detailní popis

Struktura pro paměť lexému.

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [lex_scanner.c](#)

4.16 Dokumentace struktury stack_node

uzel v zásobníku

```
#include <symstack.h>
```

Diagram tříd pro stack_node:

Datové položky

- [tree_node](#) * **symtable**
- char * **identifier**
- struct [stack_node](#) * **next**

4.16.1 Detailní popis

uzel v zásobníku

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [symstack.h](#)

4.17 Dokumentace struktury symbol_data

Data uloženého symbolu.

```
#include <symtable.h>
```

Datové položky

- [symbol_id_type](#) **id_type**
- [symbol_data_type](#) **data_type**
- char * **identifier**
- int **arity**
- int **offset**

4.17.1 Detailní popis

Data uloženého symbolu.

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [symtable.h](#)

4.18 Dokumentace struktury symstack

zásobník, ukazatel na vrchol zásobníku

```
#include <symstack.h>
```

Diagram tříd pro symstack:

Datové položky

- [stack_node](#) * **top**

4.18.1 Detailní popis

zásobník, ukazatel na vrchol zásobníku

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [symstack.h](#)

4.19 Dokumentace struktury Token

Struktura tokenu s načteným řetězcem a jeho hodnotou.

```
#include <lex_scanner.h>
```

Diagram tříd pro Token:

Datové struktry

- union [value](#)

Datové položky

- [TokenType](#) type
- char * **lexeme**
- union [Token::value](#) value

4.19.1 Detailní popis

Struktura tokenu s načteným řetězcem a jeho hodnotou.

Uvolnění paměti má na starosti parser.

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [lex_scanner.h](#)

4.20 Dokumentace struktury tree_node

Uzel stromu.

```
#include <symtable.h>
```

Diagram tříd pro tree_node:

Datové položky

- [symbol_data](#) **data**
- struct [tree_node](#) * **left**
- struct [tree_node](#) * **right**
- int **height**

4.20.1 Detailní popis

Uzel stromu.

Dokumentace pro tuto strukturu (struct) byla generována z následujícího souboru:

- [symtable.h](#)

4.21 Dokumentace unie Token::value

Datové položky

- int **int_val**
- double **float_val**
- char * **string_val**

Dokumentace pro tuto unii (union) byla generována z následujícího souboru:

- [lex_scanner.h](#)

Kapitola 5

Dokumentace souborů

5.1 Dokumentace souboru ast.c

Implementace funkcí pro tvorbu a uvolnění uzlů AST.

```
#include "ast.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
```

Graf závislostí na vkládaných souborech pro ast.c:

5.2 Dokumentace souboru ast.h

Definice struktur pro abstraktní syntaktický strom (AST) jazyka IFJ25.

```
#include "lex_scanner.h"
#include "symtable.h"
#include <stdlib.h>
```

Graf závislostí na vkládaných souborech pro ast.h: Následující graf ukazuje, které soubory přímo nebo nepřímo vkládají tento soubor:

Datové struktry

- struct [AstNode](#)
Základní struktura každého uzlu AST.
- struct [AstNodeList](#)
Obecný seznam AST uzlů.
- struct [AstNodeLiteral](#)
- struct [AstNodeVariable](#)
- struct [AstNodeBinaryExpr](#)
- struct [AstNodeFuncCall](#)
- struct [AstNodeBlock](#)
- struct [AstNodeIfStmt](#)
- struct [AstNodeWhileStmt](#)
- struct [AstNodeVarDecl](#)
- struct [AstNodeAssignStmt](#)
- struct [AstNodeReturnStmt](#)
- struct [AstNodeFuncDef](#)
- struct [AstNodeProgram](#)

IFJ25

- enum `AstNodeType` {
`AST_PROGRAM`, `AST_FUNC_DEF`, `AST_FUNC_CALL`, `AST_STMT_BLOCK`,
`AST_STMT_VAR_DECL`, `AST_STMT_ASSIGN`, `AST_STMT_IF`, `AST_STMT_WHILE`,
`AST_STMT_RETURN`, `AST_EXPR_BINARY`, `AST_EXPR_LITERAL`, `AST_EXPR_VARIABLE`,
`AST_NODE_LIST` }
- Typ uzlu v abstraktním syntaktickém stromu.*
- enum `AstFuncKind` { `FUNC_IS_FUNC`, `FUNC_IS_GETTER`, `FUNC_IS_SETTER` }
- Druh funkce (běžná funkce, getter, setter).*
- typedef struct `AstNode` `AstNode`
- Základní struktura každého uzlu AST.*
- `AstNode * create_literal (Token token)`
- Vytvoří AST uzel pro literál.*
- `AstNode * create_variable (Token token)`
- Vytvoří AST uzel pro proměnnou.*
- `AstNode * create_binary_expr (TokenType op, AstNode *left, AstNode *right, int line)`
- Vytvoří AST uzel pro binární výraz.*
- `AstNode * create_func_call (Token func_id, AstNodeList *args)`
- Vytvoří AST uzel pro volání funkce.*
- `AstNode * create_if_stmt (AstNode *cond, AstNodeBlock *then_b, AstNodeBlock *else_b, int line)`
- Vytvoří AST uzel pro příkaz if.*
- `AstNode * create_while_stmt (AstNode *cond, AstNodeBlock *body, int line)`
- Vytvoří AST uzel pro příkaz while.*
- `AstNode * create_return_stmt (AstNode *expr, int line)`
- Vytvoří AST uzel pro příkaz return.*
- `AstNode * create_vardecl (Token var_id)`
- Vytvoří AST uzel pro deklaraci proměnné.*
- `AstNode * create_assign_stmt (AstNodeVariable *lvalue, AstNode *rvalue, int line)`
- Vytvoří AST uzel pro přiřazení.*
- `AstNodeList * create_list (void)`
- Vytvoří prázdný seznam AST uzlů.*
- void `add_to_list (AstNodeList *list, AstNode *item)`
- Přidá položku na konec seznamu AST uzlů.*
- `AstNodeBlock * create_block (int line)`
- Vytvoří AST uzel pro blok příkazů.*
- void `add_stmt_to_block (AstNodeBlock *block, AstNode *stmt)`
- Přidá příkaz do bloku.*
- `AstNodeFuncDef * create_func_def (Token func_id, AstNodeList *params, AstNodeBlock *body, AstFuncKind kind)`
- Vytvoří AST uzel pro definici funkce.*
- `AstNodeProgram * create_program (void)`
- Vytvoří kořenový uzel programu.*
- void `add_func_to_program (AstNodeProgram *prog, AstNodeFuncDef *func)`
- Přidá definici funkce do kořenového uzlu programu.*
- void `free_ast_node (AstNode *node)`
- Rekurzivně uvolní uzel AST a všechny jeho potomky.*
- void `free_token_lexeme (Token *t)`
- Uvolní dynamicky alokované části tokenu (lexeme, string_val).*

5.2.1 Detailní popis

Definice struktur pro abstraktní syntaktický strom (AST) jazyka IFJ25.

Autor

Jan Kostečka xkostej00

5.2.2 Dokumentace výčtových typů

5.2.2.1 AstNodeType

enum `AstNodeType`

Typ uzlu v abstraktním syntaktickém stromu.

Hodnoty výčtu

AST_PROGRAM	Kořen stromu, obsahuje seznam funkcí
AST_FUNC_DEF	Definice funkce (static name(params) block)
AST_FUNC_CALL	Volání funkce (name(args))
AST_STMT_BLOCK	Blok kódu { ... }.
AST_STMT_VAR_DECL	var id
AST_STMT_ASSIGN	id = expr
AST_STMT_IF	if (cond) then else
AST_STMT_WHILE	while (cond) block
AST_STMT_RETURN	return expr
AST_EXPR_BINARY	expr OP expr
AST_EXPR_LITERAL	123, "abc", null
AST_EXPR_VARIABLE	id
AST_NODE_LIST	Obecný seznam (pro argumenty, parametry, příkazy)

5.2.3 Dokumentace funkcí

5.2.3.1 add_func_to_program()

```
void add_func_to_program (  
    AstNodeProgram * prog,  
    AstNodeFuncDef * func )
```

Přidá definici funkce do kořenového uzlu programu.

Parametry

<i>prog</i>	Kořenový uzel programu.
<i>func</i>	Definice funkce, která se má přidat.

5.2.3.2 add_stmt_to_block()

```
void add_stmt_to_block (
    AstNodeBlock * block,
    AstNode * stmt )
```

Přidá příkaz do bloku.

Parametry

<i>block</i>	Blok, do kterého se příkaz přidává.
<i>stmt</i>	Přidávaný příkaz (AST uzel).

5.2.3.3 add_to_list()

```
void add_to_list (
    AstNodeList * list,
    AstNode * item )
```

Přidá položku na konec seznamu AST uzlů.

Pokud je kapacita naplněna, seznam se zvětší pomocí realloc().

Parametry

<i>list</i>	Seznam, do kterého se přidává.
<i>item</i>	Přidávaný uzel AST.

5.2.3.4 create_assign_stmt()

```
AstNode* create_assign_stmt (
    AstNodeVariable * lvalue,
    AstNode * rvalue,
    int line )
```

Vytvoří AST uzel pro přiřazení.

Parametry

<i>lvalue</i>	Levá strana (proměnná).
<i>rvalue</i>	Pravá strana (výraz).
<i>line</i>	Číslo řádku, na kterém příkaz začíná.

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.5 `create_binary_expr()`

```
AstNode* create_binary_expr (  
    TokenType op,  
    AstNode * left,  
    AstNode * right,  
    int line )
```

Vytvoří AST uzel pro binární výraz.

Parametry

<i>op</i>	Typ operátoru (PLUS, MINUS, atd.).
<i>left</i>	Levý operand.
<i>right</i>	Pravý operand.
<i>line</i>	Číslo řádku, na kterém se výraz nachází.

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.6 `create_block()`

```
AstNodeBlock* create_block (  
    int line )
```

Vytvoří AST uzel pro blok příkazů.

Parametry

<i>line</i>	Číslo řádku, na kterém blok začíná.
-------------	-------------------------------------

Návratová hodnota

Ukazatel na nově vytvořený blok.

5.2.3.7 create_func_call()

```
AstNode* create_func_call (
    Token func_id,
    AstNodeList * args )
```

Vytvoří AST uzel pro volání funkce.

Parametry

<i>func_id</i>	Token s identifikátorem funkce (nebo vestavěné IFJ funkce).
<i>args</i>	Seznam argumentů volání.

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.8 create_func_def()

```
AstNodeFuncDef* create_func_def (
    Token func_id,
    AstNodeList * params,
    AstNodeBlock * body,
    AstFuncKind kind )
```

Vytvoří AST uzel pro definici funkce.

Parametry

<i>func_id</i>	Identifikátor funkce.
<i>params</i>	Seznam parametrů.
<i>body</i>	Tělo funkce.
<i>kind</i>	Druh funkce (běžná, getter, setter).

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.9 create_if_stmt()

```
AstNode* create_if_stmt (
    AstNode * cond,
    AstNodeBlock * then_b,
    AstNodeBlock * else_b,
    int line )
```

Vytvoří AST uzel pro příkaz if.

Parametry

<i>cond</i>	Podmínka.
<i>then</i> _↔ <i>_b</i>	Větev then.
<i>else</i> _↔ <i>_b</i>	Větev else (může být NULL).
<i>line</i>	Číslo řádku, na kterém příkaz začíná.

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.10 create_list()

```
AstNodeList* create_list (
    void )
```

Vytvoří prázdný seznam AST uzlů.

Seznam má počáteční kapacitu 8 položek a podle potřeby se dynamicky zvětšuje.

5.2.3.11 create_literal()

```
AstNode* create_literal (
    Token token )
```

Vytvoří AST uzel pro literál.

Vytvoří AST uzel pro literál.

Parametry

<i>token</i>	<code>Token</code> reprezentující literál.
--------------	--

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.12 create_program()

```
AstNodeProgram* create_program (
    void )
```

Vytvoří kořenový uzel programu.

Kořen obsahuje seznam definic funkcí.

Návratová hodnota

Ukazatel na nově vytvořený uzel programu.

5.2.3.13 create_return_stmt()

```
AstNode* create_return_stmt (
    AstNode * expr,
    int line )
```

Vytvoří AST uzel pro příkaz return.

Parametry

<i>expr</i>	Vracený výraz (může být NULL).
<i>line</i>	Číslo řádku, na kterém příkaz začíná.

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.14 create_vardecl()

```
AstNode* create_vardecl (
    Token var_id )
```

Vytvoří AST uzel pro deklaraci proměnné.

Parametry

<i>var</i> ↔ <i>_id</i>	Token s identifikátorem proměnné.
----------------------------	-----------------------------------

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.15 create_variable()

```
AstNode* create_variable (
    Token token )
```

Vytvoří AST uzel pro proměnnou.

Parametry

<i>token</i>	Token s identifikátorem proměnné.
--------------	-----------------------------------

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.16 create_while_stmt()

```
AstNode* create_while_stmt (
    AstNode * cond,
    AstNodeBlock * body,
    int line )
```

Vytvoří AST uzel pro příkaz while.

Parametry

<i>cond</i>	Podmínka cyklu.
<i>body</i>	Tělo cyklu.
<i>line</i>	Číslo řádku, na kterém příkaz začíná.

Návratová hodnota

Ukazatel na nově vytvořený uzel.

5.2.3.17 free_ast_node()

```
void free_ast_node (
    AstNode * node )
```

Rekurzivně uvolní uzel AST a všechny jeho potomky.

Funkce korektně uvolní i vnořené struktury (bloky, seznamy, výrazy, definice funkcí atd.) a všechny dynamicky alokované lexémy.

Parametry

<i>node</i>	Kořen uvolňovaného podstromu (může být NULL).
-------------	---

5.2.3.18 free_token_lexeme()

```
void free_token_lexeme (
    Token * t )
```

Uvolní dynamicky alokované části tokenu (lexeme, string_val).

Parametry

<i>t</i>	Ukazatel na token, jehož interní data se mají uvolnit.
----------	--

5.3 Dokumentace souboru codegen.c

generování cílového kódu IFJcode25

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include "codegen.h"
#include "ast.h"
#include "parser.h"
#include "err.h"
```

Graf závislostí na vkládaných souborech pro codegen.c:

IFJ25

- char `global_vars` [128][256]
Pole pro uložení globálních proměnných.
- int `counter` = 0
Počet globálních proměnných.
- int `if_label_counter` = 0

- Čítač pro generování unikátních návěstí (*if, while...*)
- void `gen_nil_check` ()
Generuje kontrolu na nil hodnoty pro binární operace.
- void `help_func` ()
vypíše kód vestavěných funkcí
- void `generate_code` (`AstNode` *root)
Vstupní funkce pro generování kódu z AST volání z `main.c`. Nejprve spustí první průchod pro sběr globálních proměných, potom se vypíší globální proměnné, pomocné proměnné a vestavěné funkce. Nakonec se provede hlavní průchod AST pro generování kódu.

5.3.1 Detailní popis

generování cílového kódu IFJcode25

Autor

Jáchym Turek xturekj01

Soubor obsahuje funkce pro generování kódu z AST. Generovaný kód je vypsán na standardní výstup. Prochází strom a pro každý uzel generuje odpovídající unstrukce IFJcode25.

5.3.2 Dokumentace funkcí

5.3.2.1 `generate_code()`

```
void generate_code (
    AstNode * root )
```

Vstupní funkce pro generování kódu z AST volání z `main.c`. Nejprve spustí první průchod pro sběr globálních proměných, potom se vypíší globální proměnné, pomocné proměnné a vestavěné funkce. Nakonec se provede hlavní průchod AST pro generování kódu.

Parametry

<code>root</code>	Kořen AST
-------------------	-----------

5.4 Dokumentace souboru codegen.h

generování cílového kódu IFJcode25

```
#include "ast.h"
#include "parser.h"
```

Graf závislostí na vkládaných souborech pro codegen.h: Následující graf ukazuje, které soubory přímo nebo nepřímo vkládají tento soubor:

Funkce

IFJ25

- void `generate_code` (`AstNode` *root)

Vstupní funkce pro generování kódu z AST volání z `main.c`. Nejprve spustí první průchod pro sběr globálních proměnných, potom se vypíšíou globální proměnné, pomocné proměnné a vestavěné funkce. Nakonec se provede hlavní průchod AST pro generování kódu.

5.4.1 Detailní popis

generování cílového kódu IFJcode25

Autor

Jáchym Turek xturekj01

Hlavičkový soubor pro generátor kódu.

5.4.2 Dokumentace funkcí

5.4.2.1 `generate_code()`

```
void generate_code (
    AstNode * root )
```

Vstupní funkce pro generování kódu z AST volání z `main.c`. Nejprve spustí první průchod pro sběr globálních proměnných, potom se vypíšíou globální proměnné, pomocné proměnné a vestavěné funkce. Nakonec se provede hlavní průchod AST pro generování kódu.

Parametry

<code>root</code>	Kořen AST
-------------------	-----------

5.5 Dokumentace souboru `err.c`

Soubor obsahující jednotnou funkci pro správné vypnutí programu se správnou návratovou hodnotou.

```
#include <stdio.h>
#include <stdlib.h>
#include "err.h"
```

Graf závislostí na vkládaných souborech pro `err.c`:

Funkce

IFJ25

- void `ifjexit` (int err_code)
Funkce vypne program.

5.5.1 Detailní popis

Soubor obsahující jednotnou funkci pro správné vypnutí programu se správnou návratovou hodnotou.

Autor

Jakub Jan Kupčák xkupcij00

5.5.2 Dokumentace funkcí

5.5.2.1 ifjexit()

```
void ifjexit (
    int err_code )
```

Funkce vypne program.

Parametry

<code>err_code</code>	návratová hodnota
-----------------------	-------------------

Návratová hodnota

void

5.6 Dokumentace souboru err.h

Hlavičkový soubor deklarující všechny návratové hodnoty.

Následující graf ukazuje, které soubory přímo nebo nepřímo vkládají tento soubor:

IFJ25

- `#define ERR_LEX 1`
- `#define ERR_SYN 2`
- `#define ERR_SEM_UNDEFINED 3`
- `#define ERR_SEM_REDEFINED 4`

- `#define ERR_SEM_PARAM 5`
- `#define ERR_SEM_INCOMP 6`
- `#define ERR_SEM_OTHER 10`
- `#define ERR_RSEM_PARAM 25`
- `#define ERR_RSEM_INCOMP 26`
- `#define ERR_INTERNAL 99`
- `void ifjexit (int err_code)`

Funkce vypne program.

5.6.1 Detailní popis

Hlavičkový soubor deklarující všechny návratové hodnoty.

Autor

Jakub Jan Kupčák xkupcij00

5.6.2 Dokumentace funkcí

5.6.2.1 ifjexit()

```
void ifjexit (  
    int err_code )
```

Funkce vypne program.

Parametry

<code>err_code</code>	návratová hodnota
-----------------------	-------------------

Návratová hodnota

`void`

5.7 Dokumentace souboru lex_scanner.c

Lexikální analýza.

```
#include <stdio.h>  
#include <ctype.h>  
#include <string.h>  
#include <stdbool.h>  
#include <stdlib.h>  
#include "lex_scanner.h"  
#include "err.h"
```

Graf závislostí na vkládaných souborech pro lex_scanner.c:

Datové struktry

- struct [LexemeBuffer](#)
Struktura pro paměť lexému.

IFJ25

- [Token createToken](#) ([TokenType](#) type, [LexemeBuffer](#) *lexeme)
Vytvoření tokenu.
- [LexemeBuffer](#) * [initBuffer](#) (size_t initial_size)
Přidělení paměti pro lexém.
- void [freeBuffer](#) ([LexemeBuffer](#) *lb)
Uvolnění paměti lexému.
- void [resetBuffer](#) ([LexemeBuffer](#) *lb)
Vymazání obsahu uloženého lexému.
- void [appendChar](#) ([LexemeBuffer](#) *lb, char c)
Přidání znaku do lexému.
- void [removeLast](#) ([LexemeBuffer](#) *lb)
Odebrání posledního znaku z lexému.
- char * [copyString](#) (const char *str)
Pomocná funkce pro kopírování řetězce a přiřazení paměti.
- [State builtinFunctions](#) ([LexemeBuffer](#) *lexeme)
Vytvoření tokenu vestavěné funkce.
- void [keywords](#) ([LexemeBuffer](#) *lexeme)
Vyhodnocení klíčového slova.
- char [escapeChar](#) ()
Přetypování escape sekvence.
- [State transition](#) ([State](#) state, int key, [LexemeBuffer](#) *lexeme)
Vytvoření tokenu vestavěné funkce.
- [Token ifj_get_token](#) (void)
Vrátí další token z lexikálního analyzátoru.
- int [ifj_get_line](#) (void)
Vrátí číslo aktuálního řádku.

5.7.1 Detailní popis

Lexikální analýza.

Autor

Petr Molík xmolikp00

Soubor načte znak ze standardního vstupu, pomocí konečného automatu se přesouvá do stavů a zpracovává načtené znaky. Jakmile se dostane do koncového stavu, vytvoří se token a analýza pokračuje dalším znakem ze startovního stavu.

5.7.2 Dokumentace funkcí

5.7.2.1 builtInFunctions()

```
State builtInFunctions (
    LexemeBuffer * lexeme )
```

Vytvoření tokenu vestavěné funkce.

Funkce podle načteného řetězce vytvoří token pro danou vestavěnou funkci. Pokud se token vytvořil vrací zpátky Start, jinak Error.

Viz také

[createToken\(\)](#)

Parametry

<i>lexeme</i>	Načtený řetězec
---------------	-----------------

Návratová hodnota

State

5.7.2.2 createToken()

```
Token createToken (
    TokenType type,
    LexemeBuffer * lexeme )
```

Vytvoření tokenu.

Vytvoří token s daným typem a načteným řetězcem(lexem). Některým tokenům se přiřadí i číselná hodnota nebo řetězec, který předsatvují.

Parametry

<i>type</i>	Typ tokenu
<i>lexeme</i>	Načtený řetězec tokenu

Návratová hodnota

Token

5.7.2.3 escapeChar()

```
char escapeChar ( )
```

Přetypování escape sekvence.

Pokud se jedná o escape sekvenci, vrátí funkce znak odpovídající dané sekvenci.

Návratová hodnota

char, nebo -1 při chybě/neplatné sekvenci

5.7.2.4 `ifj_get_line()`

```
int ifj_get_line (  
    void )
```

Vrátí číslo aktuálního řádku.

Vrátí aktuální číslo řádku.

5.7.2.5 `ifj_get_token()`

```
Token ifj_get_token (  
    void )
```

Vrátí další token z lexikálního analyzátoru.

Vrátí další token ze vstupu.

5.7.2.6 `keywords()`

```
void keywords (  
    LexemeBuffer * lexeme )
```

Vyhodnocení klíčového slova.

Funkce zkontroluje, jestli není načtený řetězec klíčové slovo. Pokud je, vytvoří pro něj token, jinak se vytvoří token identifikátoru.

Parametry

<code>lexeme</code>	Načtený řetězec
---------------------	-----------------

5.7.2.7 `transition()`

```
State transition (  
    State state,
```

```
int key,
LexemeBuffer * lexeme )
```

Vytvoření tokenu vestavěné funkce.

Jedná se o konečný automat. Podle vstupního stavu se funkce rozhodne do jakého stavu vstoupí. Tam se na základě načteného stavu vytvoří token (jedná se o koncový stav), nebo vrací následující stav.

Parametry

<i>state</i>	Aktuální stav
<i>key</i>	Načtený znak
<i>lexeme</i>	Paměť pro uložení řetězce tokenu

Návratová hodnota

State

5.8 Dokumentace souboru lex_scanner.h

Lexikální analýza.

Následující graf ukazuje, které soubory přímo nebo nepřímo vkládají tento soubor:

Datové struktry

- struct [Token](#)
Struktura tokenu s načteným řetězcem a jeho hodnotou.
- union [Token::value](#)

IFJ25

- enum [State](#) {
Start , Letter , Zero , Number ,
Hex , Exponent , Float , Exponent_sign ,
Exponent_float , BuiltIn , BuiltIn_dot , BuiltIn_fun ,
Divide , Lesser , Greater , Assign ,
Comment_single , Comment_multi , Comment_multi_end , Comment_multi_end_check ,
Comment_multi_slash , String_single , String_detect , String_q_check1 ,
String_q_check2 , String_multi , Id_global0 , Id_global ,
Not , Error }
- enum [TokenType](#) {
KEYWORD_class , KEYWORD_if , KEYWORD_else , KEYWORD_is ,
KEYWORD_null , KEYWORD_return , KEYWORD_var , KEYWORD_while ,
KEYWORD_ifj , KEYWORD_static , KEYWORD_import , KEYWORD_for ,
KEYWORD_Num , KEYWORD_String , KEYWORD_Null , IDENTIFIER_LOCAL ,
IDENTIFIER_GLOBAL , INT , FLOAT , STRING ,
TIMES , DIVIDE , PLUS , MINUS ,
LESSER , GREATER , LESSER_EQUAL , GREATER_EQUAL ,
EQUAL , NOT_EQUAL , ASSIGN , L_ROUND ,
R_ROUND , L_CURLY , R_CURLY , IFJ_READ_STR ,
IFJ_READ_NUM , IFJ_WRITE , IFJ_FLOOR , IFJ_STR ,
IFJ_LENGTH , IFJ_SUBSTRING , IFJ_STRCMP , IFJ_ORD ,
IFJ_CHR , COMMA , EOL , T_EOF }

- Všechny typy tokenů*
- typedef enum [State](#) [State](#)
Stavy konečného automatu.
- typedef enum [TokenType](#) [TokenType](#)
Všechny typy tokenů
- typedef struct [Token](#) [Token](#)
Struktura tokenu s načteným řetězcem a jeho hodnotou.
- [Token](#) [ifj_get_token](#) (void)
Vrátí další token ze vstupu.
- int [ifj_get_line](#) (void)
Vrátí aktuální číslo řádku.

5.8.1 Detailní popis

Lexikální analýza.

Autor

Petr Molík xmolikp00

5.8.2 Dokumentace definic typů

5.8.2.1 Token

```
typedef struct Token Token
```

Struktura tokenu s načteným řetězcem a jeho hodnotou.

Uvolnění paměti má na starosti parser.

5.8.3 Dokumentace funkcí

5.8.3.1 ifj_get_line()

```
int ifj_get_line (  
    void )
```

Vrátí aktuální číslo řádku.

Řádek se zvyšuje při čtení znaků konce řádku v lexikálním analyzátoru.

Vrátí aktuální číslo řádku.

5.8.3.2 ifj_get_token()

```
Token ifj_get_token (
    void )
```

Vrátí další token ze vstupu.

Čte znaky ze standardního vstupu a vrací postupně jednotlivé tokeny. Po dosažení konce vstupu vrací token typu T_EOF.

Vrátí další token ze vstupu.

5.9 Dokumentace souboru main.c

Hlavičkový soubor pro tabulku symbolů

```
#include <stdio.h>
#include <string.h>
#include "parser.h"
#include "ast.h"
#include "codegen.h"
```

Graf závislostí na vkládaných souborech pro main.c:

Funkce

IFJ25

- int main ()

5.9.1 Detailní popis

Hlavičkový soubor pro tabulku symbolů

Autor

Jan Kostečka xkostej00
Jakub Jan Kupčík xkupcij00
Jáchym Turek xturekj01
Petr Molík xmolikp00

5.10 Dokumentace souboru parser.c

Syntaktická analýza (rekurzivní sestup + precedenční pro výrazy) a generování AST pro jazyk IFJ25.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <assert.h>
#include <stdarg.h>
#include "lex_scanner.h"
#include "err.h"
#include "parser.h"
#include "ast.h"
#include "symtable.h"
#include "symstack.h"
```

Graf závislostí na vkládaných souborech pro parser.c:

IFJ25

- `#define TRACEF(...)`
- enum `EOL_POLICY` { `EOL_ZERO_OR_MORE` , `EOL_ONE_EXACT` }
- enum `IfjPrec` {
`PREC_LOWEST = 0` , `PREC_EQ_OP` , `PREC_IS_OP` , `PREC_REL_OP` ,
`PREC_ADD_OP` , `PREC_MUL_OP` }
- `symstack global_symstack`
Zásobník tabulek symbolů používaný při sémantické analýze.
- `Token ifj_get_token` (void)
Vrátí další token z lexikálního analyzátoru.
- bool `search_scopes` (`symstack *stack`, char *`identifier`, `symbol_data **found`)
Vyhledá symbol ve všech tabulkách symbolů na zásobníku.
- `symbol_data_type get_type` (`AstNode *node`)
Vrátí datový typ výrazu podle anotací v AST.
- `AstNode * ifj_parse_program` (void)
Vstupní bod parseru.

5.10.1 Detailní popis

Syntaktická analýza (rekurzivní sestup + precedenční pro výrazy) a generování AST pro jazyk IFJ25.

Autor

Jakub Jan Kupčák xkupcij00

Jan Kostečka xkosteji00

5.10.2 Dokumentace k definicím maker

5.10.2.1 TRACEF

```
#define TRACEF(
    ... )
```

Hodnota:

```
do           \
{             \
} while (0)
```

5.10.3 Dokumentace funkcí

5.10.3.1 get_type()

```
symbol_data_type get_type (
    AstNode * node )
```

Vrátí datový typ výrazu podle anotací v AST.

Parametry

<i>node</i>	Uzel AST představující výraz.
-------------	-------------------------------

Návratová hodnota

Datový typ výrazu, nebo Undefined, pokud není možné typ určit.

5.10.3.2 ifj_get_token()

```
Token ifj_get_token (
    void )
```

Vrátí další token z lexikálního analyzátoru.

Vrátí další token ze vstupu.

5.10.3.3 ifj_parse_program()

```
AstNode* ifj_parse_program (
    void )
```

Vstupní bod parseru.

Vstupní bod parseru pro jazyk IFJ25.

Načte celý vstup pomocí lexikálního analyzátoru, provede syntaktickou analýzu, vytvoří AST a nad ním dvouprůchodovou sémantickou analýzu. Při chybě volá [ifjexit\(\)](#) s odpovídajícím kódem chyby.

Návratová hodnota

Ukazatel na kořenový uzel AST (AST_PROGRAM). V případě chyby funkce nevrací a ukončí běh programu.

5.10.3.4 search_scopes()

```
bool search_scopes (
    symstack * stack,
    char * identifier,
    symbol_data ** found )
```

Vyhledá symbol ve všech tabulkách symbolů na zásobníku.

Prohledává tabulky od vrcholu zásobníku (nejvnitřnější scope) směrem ven.

Parametry

<i>stack</i>	Zásobník tabulek symbolů.
<i>identifier</i>	Hledaný identifikátor.
<i>found</i>	Výstupní ukazatel na nalezený záznam (pokud je nalezen).

Návratová hodnota

true, pokud byl symbol nalezen, jinak false.

5.11 Dokumentace souboru parser.h

Hlavičkový soubor pro syntaktický analyzátor (parser)

```
#include <stdbool.h>
#include "ast.h"
```

Graf závislostí na vkládaných souborech pro parser.h: Následující graf ukazuje, které soubory přímo nebo nepřímo vkládají tento soubor:

IFJ25

- int [ifj_error_code](#)
Globální kód chyby nastavovaný funkcí [ifjexit\(\)](#).
- [AstNode](#) * [ifj_parse_program](#) (void)
Vstupní bod parseru pro jazyk IFJ25.

5.11.1 Detailní popis

Hlavičkový soubor pro syntaktický analyzátor (parser)

Autor

Jakub Jan Kupčík [xkupcij00](#)

Jan Kostečka [xkostej00](#)

5.11.2 Dokumentace funkcí

5.11.2.1 ifj_parse_program()

```
AstNode* ifj_parse_program (
    void )
```

Vstupní bod parseru pro jazyk IFJ25.

Načte vstupní program, provede syntaktickou a sémantickou analýzu a vytvoří strom AST. Při chybě volá `ifjexit()` a ukončí program.

Návratová hodnota

Ukazatel na kořenový uzel AST (AST_PROGRAM).

Vstupní bod parseru pro jazyk IFJ25.

Načte celý vstup pomocí lexikálního analyzátoru, provede syntaktickou analýzu, vytvoří AST a nad ním dvouprůchodovou sémantickou analýzu. Při chybě volá `ifjexit()` s odpovídajícím kódem chyby.

Návratová hodnota

Ukazatel na kořenový uzel AST (AST_PROGRAM). V případě chyby funkce nevrací a ukončí běh programu.

5.12 Dokumentace souboru symstack.c

Zásobník tabulek určený k vytvoření nové tabulky symbolů pro vnořené bloky.

```
#include "symtable.h"
#include "symstack.h"
#include "err.h"
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
```

Graf závislostí na vkládaných souborech pro symstack.c:

Funkce

IFJ25

- void `stack_init` (`symstack` *stack)
Inicializace zásobníku.
- void `stack_push` (`symstack` *stack, char *identifier)
Vložení nového uzlu do zásobníku.
- void `stack_pop` (`symstack` *stack)
Odstranění vrcholu zásobníku a určení nového vrcholu.
- `tree_node` ** `stack_top` (`symstack` *stack)
Vracení ukazatele na kořen tabulky vrcholu zásobníku.
- void `stack_dispose` (`symstack` *stack)
Smazání zásobníku.

5.12.1 Detailní popis

Zásobník tabulek určený k vytvoření nové tabulky symbolů pro vnořené bloky.

Autor

Jakub Jan Kupčík xkupcij00

5.12.2 Dokumentace funkcí

5.12.2.1 stack_dispose()

```
void stack_dispose (
    symstack * stack )
```

Smazání zásobníku.

Parametry

<i>stack</i>	ukazatel na zásobník
--------------	----------------------

Návratová hodnota

void

5.12.2.2 stack_init()

```
void stack_init (
    symstack * stack )
```

Inicializace zásobníku.

Parametry

<i>stack</i>	ukazatel na zásobník
--------------	----------------------

Návratová hodnota

void

5.12.2.3 `stack_pop()`

```
void stack_pop (
    symstack * stack )
```

Odstranění vrcholu zásobníku a určení nového vrcholu.

Parametry

<i>stack</i>	ukazatel na stack
--------------	-------------------

Návratová hodnota

void

5.12.2.4 `stack_push()`

```
void stack_push (
    symstack * stack,
    char * identifier )
```

Vložení nového uzlu do zásobníku.

Parametry

<i>stack</i>	ukazatel na zásobník
<i>identifier</i>	identifikátor

Návratová hodnota

void

5.12.2.5 `stack_top()`

```
tree_node** stack_top (
    symstack * stack )
```

Vracení ukazatele na kořen tabulky vrcholu zásobníku.

Parametry

<i>stack</i>	ukazatel na zásobník
--------------	----------------------

Návratová hodnota

`tree_node` adresa ukazatele na kořen tabulky vrcholu zásobníku

5.13 Dokumentace souboru symstack.h

Hlavičkový soubor pro zásobník tabulek.

```
#include "symtable.h"
```

Graf závislostí na vkládaných souborech pro symstack.h: Následující graf ukazuje, které soubory přímo nebo nepřímo vkládají tento soubor:

Datové struktry

- struct `stack_node`
uzel v zásobníku
- struct `symstack`
zásobník, ukazatel na vrchol zásobníku

IFJ25

- typedef struct `stack_node` `stack_node`
uzel v zásobníku
- void `stack_init` (`symstack` *stack)
Inicializace zásobníku.
- void `stack_push` (`symstack` *stack, char *identifier)
Vložení nového uzlu do zásobníku.
- void `stack_pop` (`symstack` *stack)
Odstranění vrcholu zásobníku a určení nového vrcholu.
- `tree_node` ** `stack_top` (`symstack` *stack)
Vracení ukazatele na kořen tabulky vrcholu zásobníku.
- void `stack_dispose` (`symstack` *stack)
Smazání zásobníku.

5.13.1 Detailní popis

Hlavičkový soubor pro zásobník tabulek.

Autor

Jakub Jan Kupčák xkupcij00

5.13.2 Dokumentace funkcí

5.13.2.1 `stack_dispose()`

```
void stack_dispose (  
    symstack * stack )
```

Smazání zásobníku.

Parametry

<i>stack</i>	ukazatel na zásobník
--------------	----------------------

Návratová hodnota

void

5.13.2.2 stack_init()

```
void stack_init (
    symstack * stack )
```

Inicializace zásobníku.

Parametry

<i>stack</i>	ukazatel na zásobník
--------------	----------------------

Návratová hodnota

void

5.13.2.3 stack_pop()

```
void stack_pop (
    symstack * stack )
```

Odstranění vrcholu zásobníku a určení nového vrcholu.

Parametry

<i>stack</i>	ukazatel na stack
--------------	-------------------

Návratová hodnota

void

5.13.2.4 stack_push()

```
void stack_push (
    symstack * stack,
    char * identifier )
```

Vložení nového uzlu do zásobníku.

Parametry

<i>stack</i>	ukazatel na zásobník
<i>identifier</i>	identifikátor

Návratová hodnota

void

5.13.2.5 stack_top()

```
tree_node** stack_top (
    symstack * stack )
```

Vracení ukazatele na kořen tabulky vrcholu zásobníku.

Parametry

<i>stack</i>	ukazatel na zásobník
--------------	----------------------

Návratová hodnota

[tree_node](#) adresa ukazatele na kořen tabulky vrcholu zásobníku

5.14 Dokumentace souboru symtable.c

Tabulka symbolů implementována pomocí výškově vyváženého bin. stromu.

```
#include "symtable.h"
#include "err.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Graf závislostí na vkládaných souborech pro symtable.c:

Funkce

IFJ25

- void [tree_init](#) ([tree_node](#) **tree)
Inicializace stromu.
- int [get_height](#) ([tree_node](#) *node)
Zjištění výšky uzlu.
- void [put_height](#) ([tree_node](#) *node)
Zjištění a přiřazení výšky uzlu.

- int `get_balance` (`tree_node` *node)
Získání hodnoty vyváženosti stromu.
- void `rotate_right` (`tree_node` **tree)
Rotace uzlů směrem doprava.
- void `rotate_left` (`tree_node` **tree)
Rotace uzlů směrem doleva.
- void `balance_tree` (`tree_node` **tree)
Vyvážení stromu.
- bool `search_symbol` (`tree_node` *node, char *identifier, `symbol_data` **found)
Vyhledání daného symbolu v tabulce.
- void `insert_symbol` (`tree_node` **tree, `symbol_data` *data)
Vložení symbolu do tabulky.
- void `replace_by_rightmost` (`tree_node` *target, `tree_node` **tree)
Nahrazení uzlu nejpravějším uzlem.
- void `delete_symbol` (`tree_node` **tree, char *identifier)
Smazání symbolu z tabulky.
- void `tree_dispose` (`tree_node` **tree)
Smazání tabulky symbolů a uvolnění paměti.

5.14.1 Detailní popis

Tabulka symbolů implementována pomocí výškově vyváženého bin. stromu.

Autor

Jakub Jan Kupčík xkupcij00

5.14.2 Dokumentace funkcí

5.14.2.1 `balance_tree()`

```
void balance_tree (
    tree_node ** tree )
```

Vyvážení stromu.

Parametry

<code>tree</code>	ukazatel na ukazatel na kořen
-------------------	-------------------------------

Návratová hodnota

void

5.14.2.2 delete_symbol()

```
void delete_symbol (
    tree_node ** tree,
    char * identifier )
```

Smazání symbolu z tabulky.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen stromu
<i>identifier</i>	identifikátor

Návratová hodnota

void

5.14.2.3 get_balance()

```
int get_balance (
    tree_node * node )
```

Získání hodnoty vyváženosti stromu.

Parametry

<i>node</i>	ukazatel na uzel stromu
-------------	-------------------------

Návratová hodnota

int hodnota vyváženosti

5.14.2.4 get_height()

```
int get_height (
    tree_node * node )
```

Zjištění výšky uzlu.

Parametry

<i>node</i>	ukazatel na uzel stromu
-------------	-------------------------

Návratová hodnota

int výška uzlu

5.14.2.5 insert_symbol()

```
void insert_symbol (
    tree_node ** tree,
    symbol_data * data )
```

Vložení symbolu do tabulky.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen stromu
<i>data</i>	ukazatel na data symbolu

Návratová hodnota

void

5.14.2.6 put_height()

```
void put_height (
    tree_node * node )
```

Zjištění a přiřazení výšky uzlu.

Parametry

<i>node</i>	ukazatel na uzel stromu
-------------	-------------------------

Návratová hodnota

void

5.14.2.7 replace_by_rightmost()

```
void replace_by_rightmost (
    tree_node * target,
    tree_node ** tree )
```

Nahrazení uzlu nejpravějším uzlem.

Parametry

<i>target</i>	cílový uzel k nahrazení
<i>tree</i>	ukazatel na ukazatel na kořen stromu

Návratová hodnota

void

5.14.2.8 rotate_left()

```
void rotate_left (
    tree_node ** tree )
```

Rotace uzlů směrem doleva.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen stromu
-------------	--------------------------------------

Návratová hodnota

void

5.14.2.9 rotate_right()

```
void rotate_right (
    tree_node ** tree )
```

Rotace uzlů směrem doprava.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen stromu
-------------	--------------------------------------

Návratová hodnota

void

5.14.2.10 search_symbol()

```
bool search_symbol (
    tree_node * node,
    char * identifier,
    symbol_data ** found )
```

Vyhledání daného symbolu v tabulce.

Parametry

<i>node</i>	ukazatel na uzel
<i>identifier</i>	identifikátor symbolu
<i>found</i>	ukazatel na nalezený symbol

Návratová hodnota

bool true pokud byl symbol nalezen

5.14.2.11 tree_dispose()

```
void tree_dispose (
    tree_node ** tree )
```

Smazání tabulky symbolů a uvolnění paměti.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen
-------------	-------------------------------

Návratová hodnota

void

5.14.2.12 tree_init()

```
void tree_init (
    tree_node ** tree )
```

Inicializace stromu.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen stromu
-------------	--------------------------------------

Návratová hodnota

void

5.15 Dokumentace souboru symtable.h

Hlavičkový soubor pro tabulku symbolů

```
#include <stdbool.h>
```

Graf závislostí na vkládaných souborech pro symtable.h: Následující graf ukazuje, které soubory přímo nebo nepřímo vkládají tento soubor:

Datové struktry

- struct `symbol_data`
Data uloženého symbolu.
- struct `tree_node`
Uzel stromu.

IFJ25

- enum `symbol_id_type` { `VARIABLE_ID` , `FUNC_ID` }
Uložení symbolu jako identifikátor funkce nebo proměnné
- enum `symbol_data_type` { `Undefined` , `Null` , `String` , `Num` }
Datový typ uloženého symbolu, návratové pro funkce.
- typedef struct `tree_node` `tree_node`
Uzel stromu.
- void `tree_init` (`tree_node` **tree)
Inicializace stromu.
- bool `search_symbol` (`tree_node` *node, char *identifier, `symbol_data` **found)
Vyhledání daného symbolu v tabulce.
- void `insert_symbol` (`tree_node` **tree, `symbol_data` *data)
Vložení symbolu do tabulky.
- void `delete_symbol` (`tree_node` **tree, char *identifier)
Smazání symbolu z tabulky.
- void `tree_dispose` (`tree_node` **tree)
Smazání tabulky symbolů a uvolnění paměti.

5.15.1 Detailní popis

Hlavičkový soubor pro tabulku symbolů

Autor

Jakub Jan Kupčák xkupcij00

5.15.2 Dokumentace funkcí

5.15.2.1 delete_symbol()

```
void delete_symbol (
    tree_node ** tree,
    char * identifier )
```

Smazání symbolu z tabulky.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen stromu
<i>identifier</i>	identifikátor

Návratová hodnota

void

5.15.2.2 insert_symbol()

```
void insert_symbol (
    tree_node ** tree,
    symbol_data * data )
```

Vložení symbolu do tabulky.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen stromu
<i>data</i>	ukazatel na data symbolu

Návratová hodnota

void

5.15.2.3 search_symbol()

```
bool search_symbol (
    tree_node * node,
    char * identifier,
    symbol_data ** found )
```

Vyhledání daného symbolu v tabulce.

Parametry

<i>node</i>	ukazatel na uzel
<i>identifier</i>	identifikátor symbolu
<i>found</i>	ukazatel na nalezený symbol

Návratová hodnota

bool true pokud byl symbol nalezen

5.15.2.4 tree_dispose()

```
void tree_dispose (
    tree_node ** tree )
```

Smazání tabulky symbolů a uvolnění paměti.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen
-------------	-------------------------------

Návratová hodnota

void

5.15.2.5 tree_init()

```
void tree_init (
    tree_node ** tree )
```

Inicializace stromu.

Parametry

<i>tree</i>	ukazatel na ukazatel na kořen stromu
-------------	--------------------------------------

Návratová hodnota

void

Rejstřík

add_func_to_program
 ast.h, [19](#)
add_stmt_to_block
 ast.h, [20](#)
add_to_list
 ast.h, [20](#)
ast.c, [17](#)
ast.h, [17](#)
 add_func_to_program, [19](#)
 add_stmt_to_block, [20](#)
 add_to_list, [20](#)
 AST_EXPR_BINARY, [19](#)
 AST_EXPR_LITERAL, [19](#)
 AST_EXPR_VARIABLE, [19](#)
 AST_FUNC_CALL, [19](#)
 AST_FUNC_DEF, [19](#)
 AST_NODE_LIST, [19](#)
 AST_PROGRAM, [19](#)
 AST_STMT_ASSIGN, [19](#)
 AST_STMT_BLOCK, [19](#)
 AST_STMT_IF, [19](#)
 AST_STMT_RETURN, [19](#)
 AST_STMT_VAR_DECL, [19](#)
 AST_STMT_WHILE, [19](#)
 AstNodeType, [19](#)
 create_assign_stmt, [20](#)
 create_binary_expr, [21](#)
 create_block, [21](#)
 create_func_call, [22](#)
 create_func_def, [22](#)
 create_if_stmt, [22](#)
 create_list, [23](#)
 create_literal, [23](#)
 create_program, [24](#)
 create_return_stmt, [24](#)
 create_vardecl, [24](#)
 create_variable, [25](#)
 create_while_stmt, [25](#)
 free_ast_node, [25](#)
 free_token_lexeme, [26](#)
AST_EXPR_BINARY
 ast.h, [19](#)
AST_EXPR_LITERAL
 ast.h, [19](#)
AST_EXPR_VARIABLE
 ast.h, [19](#)
AST_FUNC_CALL
 ast.h, [19](#)
AST_FUNC_DEF
 ast.h, [19](#)
AST_NODE_LIST
 ast.h, [19](#)
AST_PROGRAM
 ast.h, [19](#)
AST_STMT_ASSIGN
 ast.h, [19](#)
AST_STMT_BLOCK
 ast.h, [19](#)
AST_STMT_IF
 ast.h, [19](#)
AST_STMT_RETURN
 ast.h, [19](#)
AST_STMT_VAR_DECL
 ast.h, [19](#)
AST_STMT_WHILE
 ast.h, [19](#)
AstNode, [7](#)
AstNodeAssignStmt, [7](#)
AstNodeBinaryExpr, [8](#)
AstNodeBlock, [8](#)
AstNodeFuncCall, [8](#)
AstNodeFuncDef, [9](#)
AstNodeIfStmt, [9](#)
AstNodeList, [9](#)
AstNodeLiteral, [10](#)
AstNodeProgram, [10](#)
AstNodeReturnStmt, [10](#)
AstNodeType
 ast.h, [19](#)
AstNodeVarDecl, [11](#)
AstNodeVariable, [11](#)
AstNodeWhileStmt, [11](#)

balance_tree
 symtable.c, [46](#)
builtInFunctions
 lex_scanner.c, [31](#)

codegen.c, [26](#)
 generate_code, [27](#)
codegen.h, [27](#)
 generate_code, [28](#)
create_assign_stmt
 ast.h, [20](#)
create_binary_expr
 ast.h, [21](#)
create_block
 ast.h, [21](#)

create_func_call
 ast.h, 22
 create_func_def
 ast.h, 22
 create_if_stmt
 ast.h, 22
 create_list
 ast.h, 23
 create_literal
 ast.h, 23
 create_program
 ast.h, 24
 create_return_stmt
 ast.h, 24
 create_vardecl
 ast.h, 24
 create_variable
 ast.h, 25
 create_while_stmt
 ast.h, 25
 createToken
 lex_scanner.c, 32

 delete_symbol
 syntable.c, 46
 syntable.h, 52

 err.c, 28
 ifjexit, 29
 err.h, 29
 ifjexit, 30
 escapeChar
 lex_scanner.c, 32

 free_ast_node
 ast.h, 25
 free_token_lexeme
 ast.h, 26

 generate_code
 codegen.c, 27
 codegen.h, 28
 get_balance
 syntable.c, 47
 get_height
 syntable.c, 47
 get_type
 parser.c, 37

 ifj_get_line
 lex_scanner.c, 33
 lex_scanner.h, 35
 ifj_get_token
 lex_scanner.c, 33
 lex_scanner.h, 35
 parser.c, 38
 ifj_parse_program
 parser.c, 38
 parser.h, 39

 ifjexit
 err.c, 29
 err.h, 30
 insert_symbol
 syntable.c, 48
 syntable.h, 52

 keywords
 lex_scanner.c, 33

 lex_scanner.c, 30
 builtInFunctions, 31
 createToken, 32
 escapeChar, 32
 ifj_get_line, 33
 ifj_get_token, 33
 keywords, 33
 transition, 33
 lex_scanner.h, 34
 ifj_get_line, 35
 ifj_get_token, 35
 Token, 35
 LexemeBuffer, 12

 main.c, 36

 parser.c, 36
 get_type, 37
 ifj_get_token, 38
 ifj_parse_program, 38
 search_scopes, 38
 TRACEF, 37
 parser.h, 39
 ifj_parse_program, 39
 put_height
 syntable.c, 48

 replace_by_rightmost
 syntable.c, 48
 rotate_left
 syntable.c, 49
 rotate_right
 syntable.c, 49

 search_scopes
 parser.c, 38
 search_symbol
 syntable.c, 49
 syntable.h, 52
 stack_dispose
 symstack.c, 41
 symstack.h, 43
 stack_init
 symstack.c, 41
 symstack.h, 44
 stack_node, 12
 stack_pop
 symstack.c, 41
 symstack.h, 44
 stack_push

- symstack.c, [42](#)
 - symstack.h, [44](#)
- stack_top
 - symstack.c, [42](#)
 - symstack.h, [45](#)
- symbol_data, [13](#)
- symstack, [13](#)
- symstack.c, [40](#)
 - stack_dispose, [41](#)
 - stack_init, [41](#)
 - stack_pop, [41](#)
 - stack_push, [42](#)
 - stack_top, [42](#)
- symstack.h, [43](#)
 - stack_dispose, [43](#)
 - stack_init, [44](#)
 - stack_pop, [44](#)
 - stack_push, [44](#)
 - stack_top, [45](#)
- symtable.c, [45](#)
 - balance_tree, [46](#)
 - delete_symbol, [46](#)
 - get_balance, [47](#)
 - get_height, [47](#)
 - insert_symbol, [48](#)
 - put_height, [48](#)
 - replace_by_rightmost, [48](#)
 - rotate_left, [49](#)
 - rotate_right, [49](#)
 - search_symbol, [49](#)
 - tree_dispose, [50](#)
 - tree_init, [50](#)
- symtable.h, [51](#)
 - delete_symbol, [52](#)
 - insert_symbol, [52](#)
 - search_symbol, [52](#)
 - tree_dispose, [53](#)
 - tree_init, [53](#)
- Token, [14](#)
 - lex_scanner.h, [35](#)
- Token::value, [15](#)
- TRACEF
 - parser.c, [37](#)
- transition
 - lex_scanner.c, [33](#)
- tree_dispose
 - symtable.c, [50](#)
 - symtable.h, [53](#)
- tree_init
 - symtable.c, [50](#)
 - symtable.h, [53](#)
- tree_node, [14](#)